테스트 계획 및 결과 보고서

LangChain 버전 마이그레이션 코드 어시스턴트

항목	내용
팀	SKN35 3rd 4Team
작성	4번 (화면 · DB · 평가) [이름]
작성일	2026-10-15
기준 코드	팀 저장소 sungwook 브랜치 (dev 최신 반영, 2026-10-14)
상태	테스트 계획 확정 · 화면 · DB · 질문 세트 검사 완료 · **비교 실험 결과 미입력**

요약

> **무엇을 만들었나**: 옛 버전 LangChain 코드를 넣고 바꿀 버전을 고르면, 대응표 · 공식 문서를 근거로 고치고 verifier가 LLM 없이 11가지를 검사해, 실패하면 최대 3회 다시 고치는 챗봇.
>
> **무엇을 시험했나**: (A) 평가 질문 36개가 올바른 시험지인지 (B) 화면 · DB가 의도대로 동작하는지 (C) 화면이 팀 그래프를 불러 실제 결과를 보여 주는지 (D) 일반 LLM과 비교해 얼마나 나은지.
>
> **지금까지 결과**: 질문 세트 검사 **36/36** · 화면 · DB 자동 점검 **19/19** · 최상위 그래프 세 갈래(변환 · 검색 · 일반) 연결 확인 · TiDB 저장 · 중복 검사 확인 · 그래프 연결 실행 **2회 모두 3회차 통과**(바뀐 6줄 모두 근거 ID) · 비교 실험 [_]

1. 개요

1-1. 배경

LangChain은 1.0에서 패키지가 나뉘고 `LLMChain`, `initialize_agent` 같은 옛 체인 · 에이전트가 빠졌습니다. 인터넷의 코드는 대부분 옛 버전이라 LLM은 옛 방식을 답하고, 틀려도 모릅니다(버전 지체 환각). 연구에서도 문서가 부족하면 LLM 코드의 42.55%만, 문서를 줘도 66.36%만 목표 버전에서 실행됩니다(When LLMs Lag Behind, 2026).

1-2. 테스트 목표

목표	확인 방법
시험지가 공정한가	옛 코드가 1.x에서 실제로 깨지는지, 기대하는 답의 import가 1.x에서 실제로 되는지 자동 확인
화면 · DB가 의도대로 동작하는가	자동 점검 봇이 로그인부터 대화 다시 열기까지 19가지를 화면으로 실행
팀 그래프와 연결되는가	같은 코드를 화면 경로로 그래프에 보내 결과 모양 · 근거 ID · 검사 결과 확인
일반 LLM보다 나은가	같은 질문 30개 · 같은 모델 · 같은 채점 기준으로 비교 (진행 예정)

1-3. 범위

포함: LangChain 0.0.300 ~ 0.3.30 코드를 1.x로 올리기(그래프), 질문에 적은 버전으로 바꾸기, LangChain 질문 · 일반 질문 답변, 화면, 대화 기록(TiDB). 제외: 바뀐 코드의 실제 실행 결과 비교.

2. 시스템 구성

단계	하는 일	담당
질문 분류	코드 변환(migration) · LangChain 질문(search) · 일반 질문(general)으로 나눔	1번
① analyzer	질문에서 코드를 뽑고 바꿀 부분 찾기 (LLM)	1번

단계	하는 일	담당
② retriever	import · 메서드 대응표 정확 매칭 + 문서 하이브리드 검색(BM25 + 벡터), 재시도 때 틀린 근거 제외	2번
③ rewriter	근거를 보고 코드 다시 쓰기, 바뀐 줄마다 근거 ID · 이유	3번
④ verifier	LLM 없이 11가지 검사, 실패하면 ②로 (최대 3회)	3번
화면 · DB · 평가	Streamlit 화면, TiDB 대화 기록, 평가 질문 · 점검 도구	4번

근거 데이터 (2번)	규모
import 대응표	9,155줄 (옛 경로 → 1.x 경로)
메서드 대응표	42개 (.run → .invoke 등)
공식 문서	최신 271개 + 옛 마이그레이션 가이드 24개
검색용 조각	11,945개 (Chroma Cloud, text-embedding-3-small)
버전별 도구 목록	23개 패키지 · 버전

verifier 11가지: ① 문법 ② 근거 ID(이번에 보여 준 근거만) ③ 변경 기록(기록 없는 변경 = 환각) ④ 남은 옛 import ⑤ import 존재 ⑥ 남은 옛 메서드(경고) ⑦ 안 쓰는 import ⑧ 반환값 보존 ⑨ 정의 안 된 이름 ⑩ 출력 꺼내기 ⑪ 체인 입출력 키

구성	선택
언어 · 패키지	Python 3.12, uv (배포는 requirements.txt)
LLM 연동	LangChain 1.4.4, LangGraph, langchain-openai (gpt-4o-mini)
벡터 DB · 검색	Chroma Cloud, BM25 + 벡터 하이브리드
DB	TiDB Cloud (users, chat_history)
화면 · 배포	Streamlit, Streamlit Community Cloud

3. 테스트 계획

3-1. 테스트 종류

구분	대상	방법	도구
A. 질문 세트 검사	평가 질문 36개	옛 코드가 1.x에서 깨지는지, 기대 import가 1.x에서 되는지 (0.x 목표는 버전별 도구 목록으로 확인)	eval/check_questions.py
B. 화면 · DB 자동 점검	화면 전체 흐름	임시 DB로 화면을 직접 실행해 19가지 확인 (유료 API 없이)	tools/bot_check.py
C. 그래프 연결 실행	화면 → 팀 그래프	같은 코드를 화면과 같은 경로로 그래프에 보내 결과 확인	src/ui/client.py run()
D. 비교 실험	일반 LLM vs 우리 챗봇	질문 30개, 같은 모델 · 같은 입력 · 같은 채점	진행 예정

3-2. 평가 질문 36개

실제 사용자가 보낼 법한 **자연어 요청 + 옛 코드**로 만들었습니다. 1.x로 올리기 30개(10유형 × 3)와 버전 오가기 6개(되돌리기 · 중간 단계)입니다. 비교 실험은 팀 그래프가 지원하는 1.x로 올리기 30개로 합니다.

유형	쉬움	중간	어려움	합계
import 경로	3	0	0	3
문서 로더 · 분할기	2	1	0	3
임베딩 · 벡터스토어	1	2	0	3
LLMChain → LCEL	0	3	0	3
체인 조합	0	1	2	3
RAG	0	2	1	3
메모리	0	2	1	3
에이전트 · 도구	0	0	3	3
출력 파서	1	2	0	3
실무 상황	0	2	1	3
합계	**7**	**15**	**8**	**30**

지금 코드 버전 분포(30개): 0.0.300 4개 · 0.0.354 3개 · 0.1.20 14개 · 0.2.17 8개 · 0.3.30 1개 · 버전 오가기 6개: 1.x → 0.2 · 0.3 · 0.1 되돌리기, 0.3 → 0.1 · 0.0.354, 0.0.300 → 0.2

3-3. 비교 실험 설계

항목	규칙
비교 대상	일반 LLM(같은 모델에 질문만) vs 우리 챗봇(팀 그래프)
모델	양쪽 같은 **[모델명]**, temperature 0

항목	규칙
입력	같은 요청 문장 + 같은 코드
채점	verifier와 같은 검사 기준 (11가지 중 오류 항목이 없으면 통과)
최대 시도	우리 챗봇 3회, 일반 LLM 1회
공정성	같은 질문 · 같은 모델 · 같은 채점기, LangChain import를 지워서 통과하는 꼼수는 실패 처리

지표	정의
검증 통과율	통과한 질문 수 ÷ 30
근거 포함률	근거 ID가 붙은 바뀐 줄 수 ÷ 전체 바뀐 줄 수
재시도 효과	최종 통과율 − 1회차 통과율 (%p)
평균 시도	통과한 질문의 시도 횟수 평균

4. 테스트 결과

4-1. A. 질문 세트 검사: 36/36 통과

36개 모두 옛 코드가 LangChain 1.4.4에서 실제로 깨지고, 기대하는 답의 import가 1.x에서 됩니다. 패키지가 없는 import는 2번의 버전별 도구 목록으로 확인했습니다. 0.x로 되돌리는 6개는 옛 경로가 목표 버전에 없는지, 기대 경로가 목표 버전에 있는지 도구 목록으로 확인했습니다. (2026-10-14)

4-2. B. 화면 · DB 자동 점검: 19/19 통과

영역	점검 항목	확인 내용	결과
로그인 · 아이디	가입		통과
	같은 아이디 가입 막기 (중복 검사)		통과
	아이디 규칙 (2~20자)		통과
	없는 아이디 로그인 막기		통과
세션 저장	한 세션에 질문 3개 이어서 저장 (질문+답 = 한 행)	세션 1개 · chat_history 3행	통과
	새 대화 → 새 세션	세션 2개	통과
버전 (질문에서 읽음)	버전 고르기 없음		통과
	시기 안내	0.0.300 전용 · 안내: 0.0.300** 시기	통과
	시기 안내	0.0.354 전용 · 안내: 0.0.354** 시기	통과
	시기 안내	0.1.20 이후 · 안내: 0.1.20 ~ 0.3.30	통과
	시기 안내	0.2.17 이후 · 안내: 0.2.17 ~ 0.3.30	통과
	시기 안내	흔한 코드 · 안내: 0.0.300 ~ 0.3.30	통과
질문 실행 · 예제 · 코드 없는 질문	질문 36개 전부 실행	오류 없음	통과
	코드 붙여 넣기 → 결과 패널		통과
	확인용 예제 30개	보내기 · 30개	통과
	코드 없는 질문 안내		통과
대화 기록 · 사용자 분리	대화 목록 저장	43개	통과
	대화 다시 열기 (질문 · 답 복원)		통과
	다른 사용자 대화 분리		통과

팀 저장소 구조 + 2번 데이터로 실행. 임시 SQLite를 써서 실제 TiDB에는 쓰지 않음. 2026-10-15 버전 선택 상자를 없애면서 (바꿀 버전은 그래프가 질문에서 읽음) 버전 선택 관련 항목을 빼고 '버전 고르기 없음' 항목을 넣어 30 → 19가지가 됐습니다.

4-3. TiDB 저장 확인 (직접 확인)

확인	결과
TiDB Cloud에 langchain_db · users · chat_history 생성	완료
화면 로그인 화면 표시 '대화 기록 저장: TiDB'	확인
가입 → users 1행, 같은 대화에 질문 2개 → chat_history 같은 session_id 2행	확인
같은 아이디 다시 가입 → '이미 있는 아이디예요'	확인
TiDB 값이 비었을 때 → 로컬 저장으로 넘어가고 이유 표시	확인

4-4. C. 그래프 연결 실행

0.0.300 시절 LLMChain 코드를 화면과 같은 경로(`run()`)로 팀 그래프에 보냈습니다. 같은 코드를 dev 반영 전후 두 번 실행해 **두 번 모두 3회차에 검사를 통과**했습니다(약 44초 · 61초).

```
from langchain.chat_models import ChatOpenAI
from langchain.prompts import PromptTemplate
from langchain.chains import LLMChain

llm = ChatOpenAI(temperature=0)
chain = LLMChain(llm=llm,
prompt=PromptTemplate.from_template('{word}의 반대말은?'))
print(chain.run(word='크다'))
```

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import
PromptTemplate
from langchain_core.output_parsers import
StrOutputParser

llm = ChatOpenAI(temperature=0)
chain = PromptTemplate.from_template('{word}의 반대말은?') | llm | StrOutputParser()
print(chain.invoke({'word': '크다'}))
```

옛 줄	새 줄	근거 ID	이유
`from langchain.chat_models import ChatOpenAI`	`from langchain_openai import ChatOpenAI`	imap-02229b7d73	ChatOpenAI import 경로 변경
`from langchain.prompts import PromptTemplate`	`from langchain_core.prompts import PromptTemplate`	imap-8ed4309a81	PromptTemplate import 경로 변경
`from langchain.chains import LLMChain`	(삭제)	imap-41c19d624a	LCEL로 바꿔 쓰지 않게 된 import 삭제
(새 줄)	`from langchain_core.output_parsers import StrOutputParser`	doc-1f037eaf7dfb	StrOutputParser 사용을 위해 추가
`chain = LLMChain(llm=llm, prompt=…)`	`chain = PromptTemplate.from_template(…) | llm | StrOutputParser()`	mmap-246a7c2258, doc-1f037eaf7dfb	LLMChain → LCEL
`print(chain.run(word='크다'))`	`print(chain.invoke({'word': '크다'}))`	mmap-246a7c2258, doc-1f037eaf7dfb	.run → .invoke, 문자열 반환 유지

바뀐 6줄 모두 근거 ID가 붙었고(근거 포함률 100%, 이 사례 기준), 근거 부족으로 못 바꾼 줄은 0줄, 설치 안내는 `langchain-core`, `langchain-openai`였습니다.

4-5. C-2. 최상위 그래프 세 갈래 연결 (2026-10-15)

질문	분류	결과
"안녕하세요"	general	2.5초, 일반 답변
"0.3에서 1.0으로 오면서 LLMChain은 어떻게 바뀌었어?"	search	12.5초, 문서 근거로 설명
옛 코드 + "최신 버전으로 바꿔 줘"	migration	16초, 목표 1.4.5 (질문에서 읽음), 1회 만에 검사 통과
새 코드 + "0.2에서 돌아가게 바꿔 줘"	migration	28초, 목표 0.2.17은 맞게 읽었으나 3회 모두 검사 실패 (5장 참고)

화면은 그래프 단계(질문 분류 → 바꿀 부분 → 근거 → 고치기 → 검사, 실패하면 다시)를 실시간으로 보여 주고, 검사에 실패하면 "같은 요청으로 다시 시도" 버튼을 띄웁니다.

4-6. D. 비교 실험 결과 [실험 후 입력]

시스템	검증 통과율	쉬움	중간	어려움	근거 포함률	평균 시도
우리 챗봇	[__]%	[__]%	[__]%	[__]%	[__]%	[__]회
일반 LLM	[__]%	[__]%	[__]%	[__]%	-	1회

1회차 통과율 **[__]%** → 최종 **[__]%** (재시도 효과 **+[__]%p**)

5. 테스트 중 발견한 문제와 조치

문제	원인	조치
대화가 질문마다 다른 세션으로 저장됨	예시 질문 버튼이 빈 대화 화면에만 있어 질문마다 '새 대화'를 눌러야 했음	새 대화를 누를 때만 세션 생성, 화면 위에 세션 정보 표시, 자동 점검 항목 추가
세션 시작 시각이 9시간 이르게 보임	DB 시각은 UTC로 저장	화면 표시만 한국 시간(+9)으로
평가 질문 1개가 1번 그래프 시험 질문과 똑같음	같은 예제를 각자 사용	겹치지 않는 새 LLMChain 질문으로 교체 (유형별 3개 유지)
팀 저장소 app.py를 화면과 1번 시험 코드가 같이 씀	같은 파일 이름	화면 입구를 streamlit_app.py로 분리, app.py는 1번 것 그대로
팀 ERD 확정 (users, chat_history)	3테이블 설계와 다름	ERD 그대로 저장 구조 변경, 아이디 중복 검사 추가 (이전 설계는 문서에 보관)
TiDB 값이 비었거나 틀리면 화면이 멈춤	연결 실패를 처리하지 않음	로컬 저장으로 넘어가고 사이드바에 이유 표시
배포 화면에서 그래프 · TiDB 연결 안 됨	Secrets 값을 읽기 전에 그래프를 불러오고, 실패를 기억함	Secrets를 먼저 읽고 키부터 확인, 실패는 기억하지 않고 다시 시도 (배포와 같은 조건에서 연결 확인)
새 입구 supervisor()가 아직 연결 중	분류 결과 이름 · 노드 연결 · 입력 형식이 맞지 않음 (1번 작업 중)	1번이 수정 (10-15 dev), 화면 입구를 supervisor()로 변경
버전 선택 상자가 그래프와 중복	그래프가 질문에서 목표 버전을 읽게 됨 (2번 target_version)	선택 상자 제거, 배지는 '원래 버전 → 바꾼 버전'
변환에 약 1분, 그동안 화면이 멈춘 것처럼 보임	검사 · 재시도 때문에 단계가 많음	그래프 진행 소식(stream)으로 단계별 진행 실시간 표시
'다시 시도해줘'라고 쳐도 엉뚱한 답	화면이 마지막 메시지만 보내서 그래프가 이전 코드를 모름	검사 실패 결과에 '같은 요청으로 다시 시도' 버튼
요청 코드에 없던 줄 (ChatCerebras 등)이 생겨 3회 모두 실패	검색이 상관없는 채팅 모델 연결 문서를 근거로 넘기고, 다시 쓰기가 그 예시를 베낌. 재시도 때 틀린 근거가 빠지지 않음	검사기가 '기록 없는 변경(환각)'으로 정확히 잡음. 2 · 3번에 검색 필터 · 다시 쓰기 규칙 · 재시도 제외 규칙 전달

6. 한계와 다음 단계

한계	다음 단계
11가지 코드 검사까지 하지만, 바뀐 코드의 실제 실행 결과는 확인하지 않음	가짜 LLM을 넣어 코드 전체 실행 테스트
옛 버전으로 되돌리기는 아직 검사 통과율이 낮고, 상관없는 문서 예시를 베끼는 경우가 있음	다시 쓰기 규칙 강화, 틀린 근거를 다음 시도에서 빼기, 상관없는 문서 걸러내기 (2 · 3번)
검사 · 재시도로 한 번 변환에 약 1분 (진행 단계는 화면에 실시간 표시)	쉬운 import는 대응표만으로 바로 답하기

한계	다음 단계
다시 연 대화는 결과 패널 없이 질문 · 답만 (ERD에 결과 열 없음)	필요하면 결과 열 추가를 팀과 협의
질문 30개 · LangChain만 대상	실사용 코드로 확장, 다른 라이브러리로 확장

부록 A. 평가 질문 36개

ID	유형	난이도	버전	요청
q01	import 경로	쉬움	0.1.20 → 1.x	회사 서버가 langchain 1.x로 올라가서 이 코드가 안 돌아가요. import만 고쳐 주세요.
q02	import 경로	쉬움	0.0.354 → 1.x	메시지 객체 import에서 에러가 나요. 최신 경로로 바꿔 주세요.
q03	import 경로	쉬움	0.3.30 → 1.x	Document 만들어서 쓰는 부분인데 0.3 코드예요. 1.x로 바꿔 주세요.
q04	문서 로더 · 분할기	쉬움	0.1.20 → 1.x	PDF 읽어서 자르는 전처리 코드인데 최신 버전에 맞게 고쳐 주세요.
q05	문서 로더 · 분할기	쉬움	0.0.354 → 1.x	웹페이지 크롤링해서 글자 수 기준으로 자르는 코드예요. 1.x로 옮겨 주세요.
q06	문서 로더 · 분할기	중간	0.2.17 → 1.x	폴더 안 txt 파일을 전부 읽어 오는 코드인데, 어디가 바뀌었는지도 같이 알려 주세요.
q07	임베딩 · 벡터스토어	쉬움	0.1.20 → 1.x	FAISS로 벡터 DB 만드는 코드가 import 에러가 나요.
q08	임베딩 · 벡터스토어	중간	0.2.17 → 1.x	검색기(retriever) 쓰는 부분까지 최신 방식으로 바꿔 주세요. get_relevant_documents가 경고가 떠요.
q09	임베딩 · 벡터스토어	중간	0.2.17 → 1.x	허깅페이스 임베딩 쓰는 코드인데 패키지가 바뀌었다고 들었어요. 뭘 설치해야 하는지도 알려 주세요.
q10	LLMChain → LCEL	중간	0.1.20 → 1.x	회의록 요약 체인인데 결과에서 ['text']를 꺼내 쓰고 있어요. 1.x에서도 문자열로 받게 해 주세요.
q11	LLMChain → LCEL	중간	0.0.300 → 1.x	predict랑 apply 쓰는 옛날 체인이에요. 여러 입력을 한 번에 돌리는 부분도 살려 주세요.
q12	LLMChain → LCEL	중간	0.1.20 → 1.x	프롬프트 결과를 바로 JSON으로 받는 체인인데 1.x 스타일로 정리해 주세요.
q13	체인 조합	중간	0.0.354 → 1.x	SimpleSequentialChain으로 두 단계 이어 붙인 코드예요. 최신 방식으로 바꿔 주세요.
q14	체인 조합	어려움	0.2.17 → 1.x	SequentialChain에서 중간 결과 여러 개를 같이 넘기는데, 이 출력 키들을 유지하면서 바꿔 주세요.
q15	체인 조합	어려움	0.1.20 → 1.x	주제에 따라 다른 프롬프트로 보내는 라우터 체인이에요. 1.x에서 어떻게 해야 해요?
q16	RAG	중간	0.1.20 → 1.x	사내 문서 QA 봇이 RetrievalQA로 돼 있어요. 1.x 방식으로 바꿔 주세요.
q17	RAG	어려움	0.2.17 → 1.x	대화 기록을 기억하는 문서 QA예요(ConversationalRetrievalChain). 기록 유지 기능은 꼭 살려 주세요.
q18	RAG	중간	0.1.20 → 1.x	load_qa_chain으로 문서 몇 개 넣고 답하는 코드예요. stuff 방식 그대로 유지해 주세요.

ID	유형	난이도	버전	요청
q19	메모리	중간	0.0.300 → 1.x	ConversationChain으로 만든 상담 챗봇인데, 대화 기억은 그대로 두고 1.x로 바꿔 주세요.
q20	메모리	어려움	0.1.20 → 1.x	최근 3턴만 기억하는 창 메모리를 쓰고 있어요. 같은 동작을 1.x에서 어떻게 하죠?
q21	메모리	중간	0.2.17 → 1.x	세션별로 대화 기록을 따로 저장해야 해요. 지금 코드를 최신 방식으로 바꿔 주세요.
q22	에이전트 · 도구	어려움	0.0.300 → 1.x	initialize_agent로 만든 계산기 에이전트예요. 1.x에서 에이전트 만드는 방식으로 바꿔 주세요.
q23	에이전트 · 도구	어려움	0.1.20 → 1.x	AgentExecutor랑 create_openai_tools_agent 쓰는 코드예요. 최신 권장 방식으로 바꿔 주세요.
q24	에이전트 · 도구	어려움	0.2.17 → 1.x	도구 호출 에이전트에 대화 기억까지 붙인 코드예요. 기억 기능 포함해서 1.x로 바꿔 주세요.
q25	출력 파서	중간	0.1.20 → 1.x	ResponseSchema로 결과를 dict로 받는 코드예요. 요즘 방식(구조화 출력)으로 바꿔 주세요.
q26	출력 파서	중간	0.2.17 → 1.x	Pydantic 파서 쓰는 코드인데 langchain.pydantic_v1에서 에러가 나요.
q27	출력 파서	쉬움	0.1.20 → 1.x	쉼표로 구분된 목록 파서예요. 경로만 바뀐 건지 확인하고 고쳐 주세요.
q28	실무 상황	중간	0.1.20 → 1.x	실행하니까 ImportError: cannot import name 'ChatOpenAI' from 'langchain.chat_models' 라고 떠요. 왜 그런지 설명하고 고쳐 주세요.
q29	실무 상황	중간	0.0.300 → 1.x	LLM 응답 캐시를 켜 두는 설정 코드예요. 전역 캐시 설정 방식이 바뀌었다는데 고쳐 주세요.
q30	실무 상황	어려움	0.1.20 → 1.x	긴 보고서를 map_reduce로 요약하는 배치 코드예요. 결과는 같게, 코드는 1.x로 바꿔 주세요. 꼭 못 바꾸는 부분이 있으면 알려 주세요.
q31	버전 오가기	중간	1.x → 0.2.17	운영 서버가 langchain 0.2에 고정돼 있어요. 1.x로 짠 이 코드를 0.2에서 돌게 되돌려 주세요.
q32	버전 오가기	어려움	1.x → 0.3.30	create_agent로 만든 에이전트인데, 고객사가 0.3을 써서 0.3에서 돌아가게 해 주세요. 안 되는 부분은 대안을 알려 주세요.
q33	버전 오가기	중간	1.x → 0.1.20	1.x에서 langchain_classic으로 겨우 돌려 둔 에이전트 코드예요. 사내 표준이 0.1이라 0.1 방식으로 되돌려 주세요.
q34	버전 오가기	중간	0.3.30 → 0.1.20	0.3에서 init_chat_model로 모델을 고르게 해 뒀는데, 0.1 서버로 옮겨야 해요. 안 되는 게 있으면 바꿔 쓰는 방법도요.
q35	버전 오가기	어려움	0.3.30 → 0.0.354	신입 교육 자료가 0.0.354 기준이라, 0.3 코드를 그 버전으로 맞춰야 해요.
q36	버전 오가기	중간	0.0.300 → 0.2.17	0.0.300 코드인데 한 번에 1.x는 부담이라 일단 0.2까지만 올리고 싶어요. 0.2에서 안 되는 것만 알려 주세요.

부록 B. 재현 방법

```
# 팀 저장소 (sungwook) 에서
uv sync
uv pip install -r requirements.txt

# A. 질문 세트 검사 (LangChain 1.x 가 깔린 별도 환경 필요)
python eval/check_questions.py --py <1.x python> --map data/processed/import_map.jsonl --symbols
data/raw/symbols

# B. 화면 · DB 자동 점검 (임시 SQLite, 유료 API 없음)
python tools/bot_check.py

# 화면 실행 (.env 에 OPENAI_API_KEY · CHROMA_* · TIDB_* )
streamlit run streamlit_app.py
```